Heart Disease Risk Prediction

An end-to-end MLOps pipeline: tracking, CI/CD, containers, Kubernetes and monitoring

MLOps (AIMLCZG523) - Assignment 01 Report

Student	Johan Varghese
BITS ID	2025AE05893
Date	19 October 2026
Dataset	UCI Heart Disease (Cleveland), 303 patients, 13 features + binary target
Code repository	https://github.com/JohanVar05893/heart-disease-monitor-mlops
Pipeline video	Watch the recorded pipeline walkthrough

1. Overview and architecture

This project builds a classifier that predicts the presence of heart disease from routine clinical measurements and wraps it in a reproducible MLOps workflow. The model is trained and tracked with MLflow, verified by automated tests and a GitHub Actions pipeline, packaged as a Docker image behind a FastAPI service, deployed on Kubernetes, and observed with Prometheus and Grafana. The emphasis is on the engineering around the model: every stage can be re-run from a clean checkout and is evidenced by the screenshots in this report.

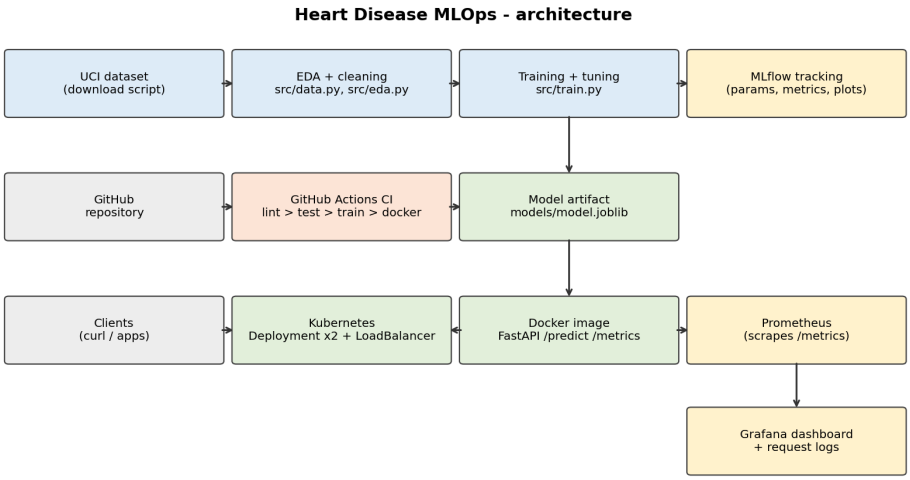


Figure 1. System architecture, from data to monitored deployment.

- **Data and training:** src/data.py, src/features.py, src/train.py; runs logged to MLflow.
- **Serving:** app/main.py (FastAPI): /predict, /health, /metrics; packaged by the Dockerfile.
- **Automation:** GitHub Actions: lint, tests, EDA, training, inference check, Docker smoke test.
- **Deployment:** Kubernetes Deployment (2 replicas) + LoadBalancer Service; Compose for monitoring.
- **Observability:** request logging, Prometheus metrics and a provisioned Grafana dashboard.

2. Setup and installation

Prerequisites: Python 3.11, Git, Docker, and kubectl with a local cluster (Docker Desktop Kubernetes, Minikube or kind). The deployment in this report was run in a GitHub Codespace using kind.

```
git clone https://github.com/JohanVar05893/heart-disease-monitor-mlops.git
cd heart-disease-monitor-mlops
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Stage	Command	Result
Data	python -m src.data	Downloads the UCI file and writes data/heart_clean.csv
EDA	python -m src.eda	Six figures in screenshots/eda/
Train + track	python -m src.train	Tunes two models, logs to MLflow, saves models/model.joblib
MLflow UI	mlflow ui --backend-store-uri sqlite:///mlflow.db	Experiment dashboard at http://127.0.0.1:5000
Inference	python -m src.predict --json '{...}'	Prediction, label and confidence
Quality	flake8 src app tests docs | pytest -v	Lint clean; 13 tests pass
Docker	docker build -t heart-api:1.0 . && docker run -p 8000:8000 heart-api:1.0	API on port 8000
Kubernetes	kubectl apply -f k8s/	2 pods behind a LoadBalancer Service
Monitoring	cd monitoring && docker compose up --build -d	API, Prometheus (9090), Grafana (3000)

A Makefile provides shortcuts for these commands, and the README documents each step.

3. Data acquisition and exploratory analysis

The UCI Heart Disease data (Cleveland subset) is downloaded by src/data.py from the UCI repository. Cleaning converts the '?' marker to a missing value, collapses the original 0-4 diagnosis scale into a binary target (0 = no disease, 1 = disease) and removes duplicate rows. The result has 303 patients: 164 without disease (54%) and 139 with disease (46%). Only ca (4 rows) and thal (2 rows) have missing values.

Missing values are kept in the cleaned CSV and imputed inside the model pipeline, so imputation statistics are learned from training folds only and cannot leak information from the test data.

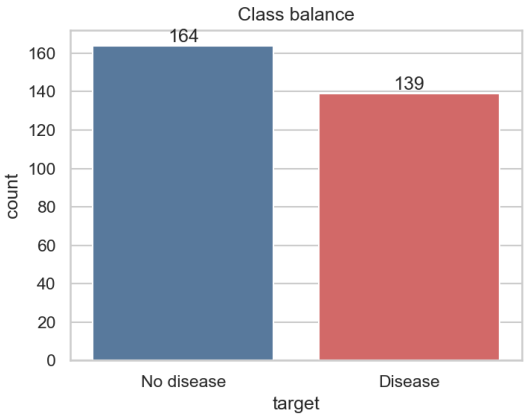


Figure 2. Class balance: near-balanced, so accuracy is meaningful and no resampling is needed.

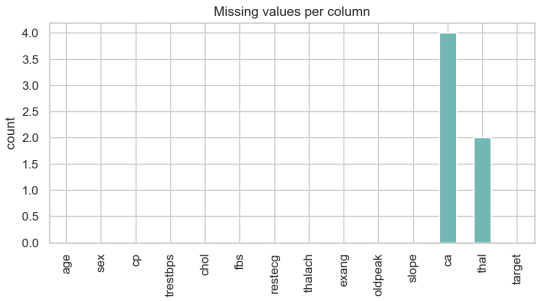


Figure 3. Missing values per column (only ca and thal).

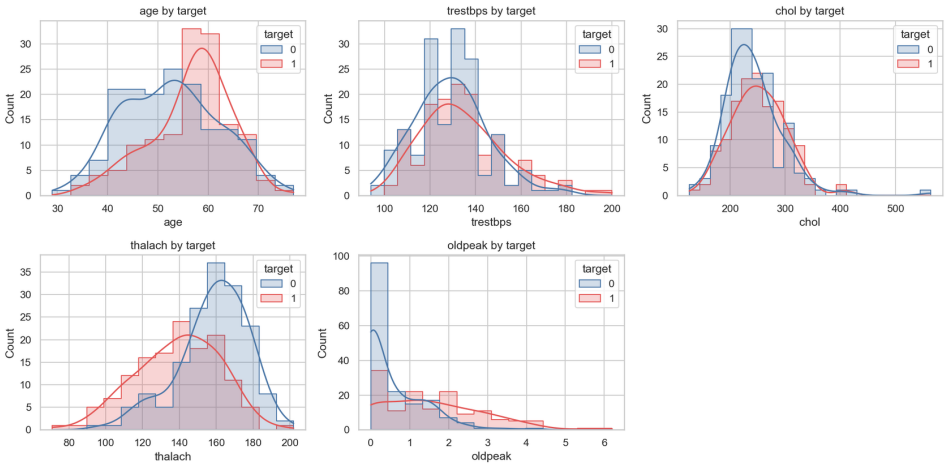


Figure 4. Distributions of the numeric features split by target.

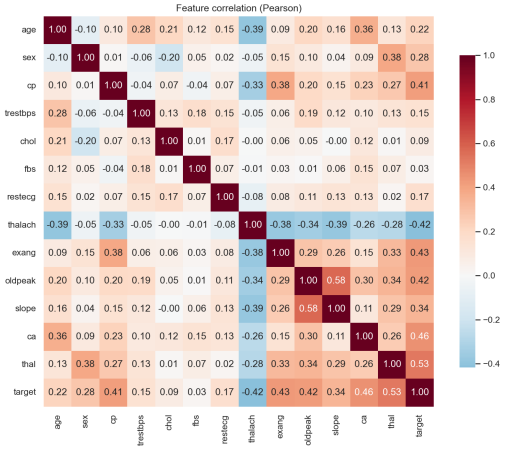


Figure 5. Pearson correlation heatmap including the target.

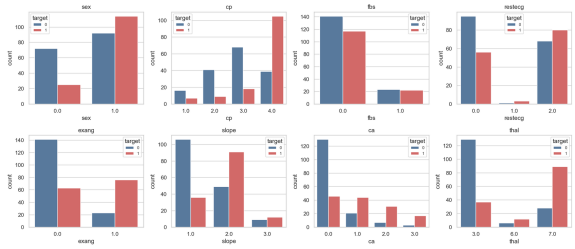


Figure 6. Categorical features against the target.

EDA findings

- thal, ca, oldpeak, exang and cp show the strongest positive correlation with disease (0.41 to 0.53).
- thalach (maximum heart rate) is the strongest negative correlation with disease (-0.42).
- Predictors are only moderately correlated with each other (largest pair: oldpeak/slope at 0.58), so multicollinearity is not a serious concern for Logistic Regression.
- Numeric features sit on very different scales, so standardisation is part of the pipeline.

4. Feature engineering and model development

Preprocessing. Numeric columns (age, trestbps, chol, thalach, oldpeak) receive median imputation and standardisation. Categorical columns (sex, cp, fbs, restecg, exang, slope, ca, thal) receive most-frequent imputation and one-hot encoding with unknown categories ignored. Preprocessing and classifier form one scikit-learn Pipeline, which is the object that is tuned, saved and served, so training-time and serving-time transformations are identical.

Training and tuning. The data is split 80/20 with stratification (seed 42). On the training portion each model is tuned with GridSearchCV using 5-fold stratified cross-validation and ROC-AUC as the selection metric. The hold-out set is used only for reporting.

Model	Hyperparameter grid
Logistic Regression	C in {0.01, 0.1, 1, 10}
Random Forest	n_estimators in {100, 300}, max_depth in {3, 5, None}, min_samples_leaf in {1, 3}

Model	Best params	CV acc	CV prec	CV recall	CV AUC	Test acc	Test prec	Test recall	Test AUC
Logistic Regression	C = 1	0.847	0.877	0.783	0.902	0.885	0.839	0.929	0.966
Random Forest	100 trees, depth 5, leaf 3	0.806	0.811	0.764	0.902	0.869	0.833	0.893	0.948

Selection. The two models tie on cross-validated ROC-AUC (0.902 to three decimals), so the tie is broken on the other cross-validated metrics. Logistic Regression is better on accuracy, precision and recall, is simpler, faster to serve and easier to interpret, so it is promoted to models/model.joblib. In a screening setting recall matters, and its hold-out recall of 0.93 is also the higher of the two.

Limitations. With only 303 patients the hold-out figures have wide confidence intervals and should be read as indicative. The data comes from a single site, so performance on other populations is unknown.

5. Experiment tracking with MLflow

Training logs every run to a local MLflow backend (SQLite file mlflow.db) in the experiment heart-disease. Each run records the model type and best hyperparameters, cross-validated and hold-out metrics with their standard deviations, the ROC curve and confusion matrix as figures, and the fitted pipeline as a model artifact. The runs table shows four runs: each model was trained once from src/train.py and once from the training notebook.

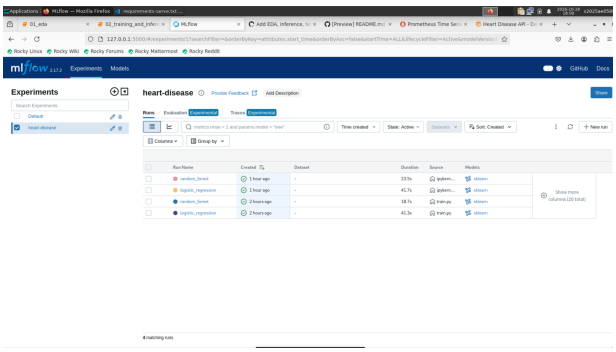


Figure 7. MLflow experiment heart-disease and its four runs.

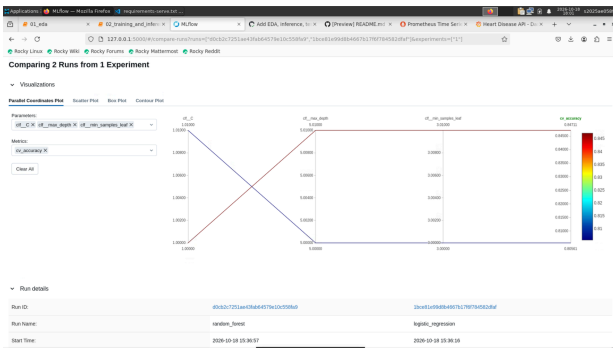


Figure 8. Side-by-side comparison of the model runs.

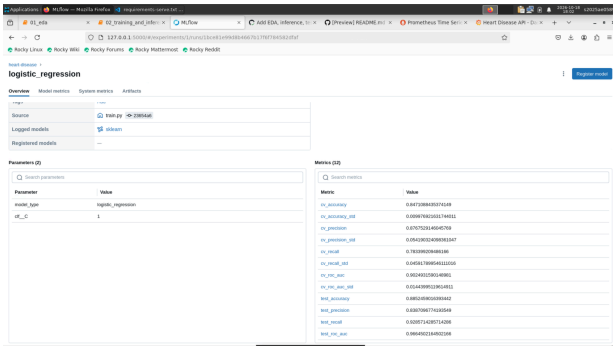


Figure 9. Logged metrics for a run.

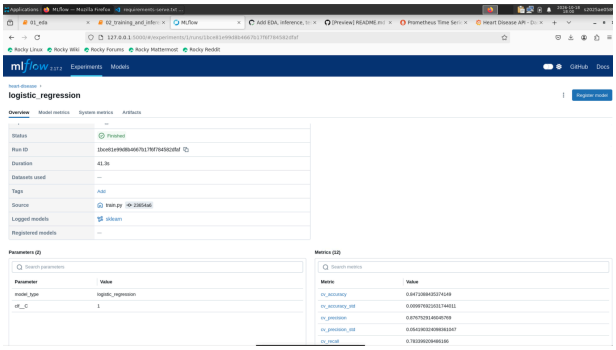


Figure 10. Selected Logistic Regression run (parameters, metrics, artifacts).

6. Model packaging and reproducibility

- **Saved model:** models/model.joblib is the full preprocessing + classifier pipeline; models/metrics.json holds its metrics.
- **Dependencies:** requirements.txt pins exact versions; requirements-serve.txt pins the smaller serving set.
- **Determinism:** fixed seeds, a stratified split, and the cleaned dataset committed to the repository.
- **Clean setup:** CI installs from requirements.txt on a fresh runner and retrains on every push; src/predict.py scores new data offline.

7. CI/CD pipeline and automated testing

The workflow .github/workflows/ci.yml runs on every push and pull request. The first job executes, in order: flake8 lint, pytest (with a JUnit report), EDA figure generation, model training, an inference smoke test and artifact upload. A second job builds the Docker image and smoke-tests /health and /predict. A failure in any step fails the workflow and the later jobs do not run.

Test file	What it checks
test_data.py, test_clean.py	Cleaned schema, binary target, no duplicates, missing-value handling
test_model.py, test_train.py	Pipeline beats chance; a full training run produces metrics and a loadable model
test_predict.py, test_eda.py	Inference output and CLI (JSON, CSV), missing-column errors, EDA figures written
test_api.py	/health, /predict, input validation (HTTP 422) and the /metrics endpoint

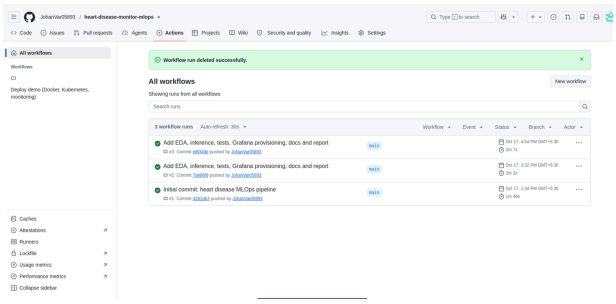


Figure 11. GitHub Actions run history: all CI runs on main succeeded.

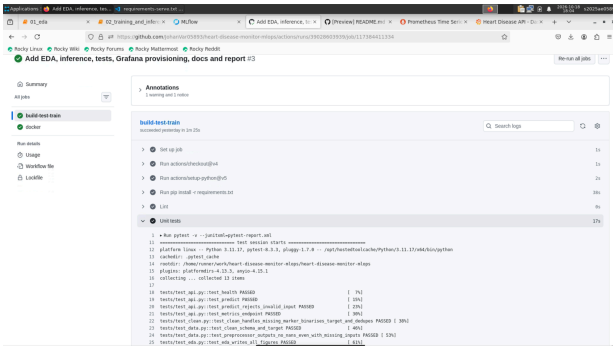


Figure 12. Unit-test step in the CI job.

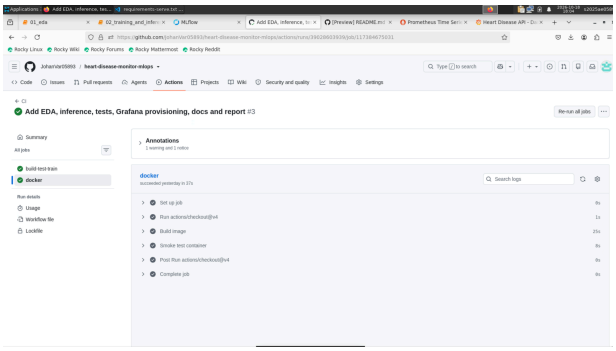


Figure 13. Docker job: image build and container smoke test.

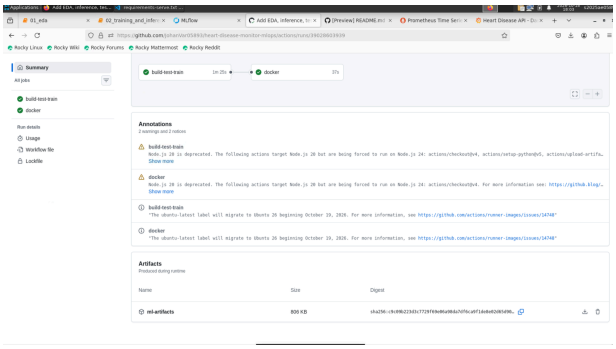


Figure 14. Artifacts uploaded by a run (models, MLflow data, EDA figures, test report).

Demonstrating that the pipeline fails on errors

To show that the pipeline blocks bad code, a deliberate lint error (an unused import) was pushed on a throwaway branch, ci-failure-demo. The build-test-train job failed at the lint step with exit code 1 and the dependent docker job was skipped. The change was never merged into main.

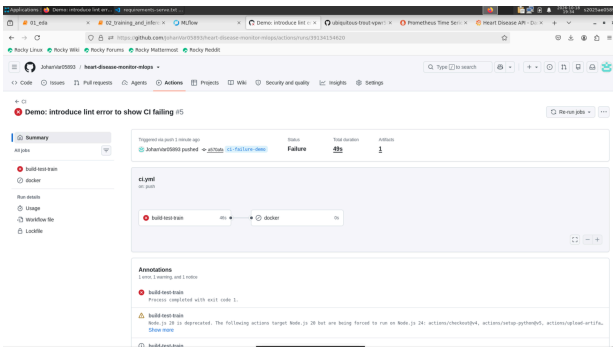


Figure 15. Failed run on the demo branch: build-test-train fails, docker is skipped.

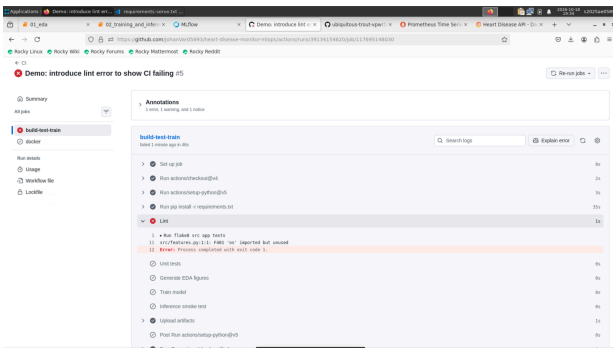


Figure 16. Failing lint step log with the flake8 error message.

8. Model containerisation

The Dockerfile uses python:3.11-slim, installs only the serving dependencies, copies the source and the trained model, defines a health check and starts uvicorn on port 8000. POST /predict accepts the 13 clinical features as JSON (ca and thal are optional) and returns the prediction, a label, the confidence in the predicted class and the disease probability. Invalid input, such as a negative age, returns HTTP 422.

```
POST /predict
{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,...}
200 OK  {"prediction":0,"label":"no heart
disease","confidence":0.8345,"probability_disease":0.1655}
```

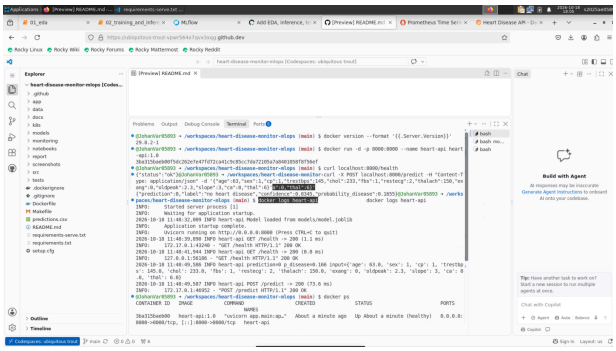


Figure 17. Docker image build and container start.

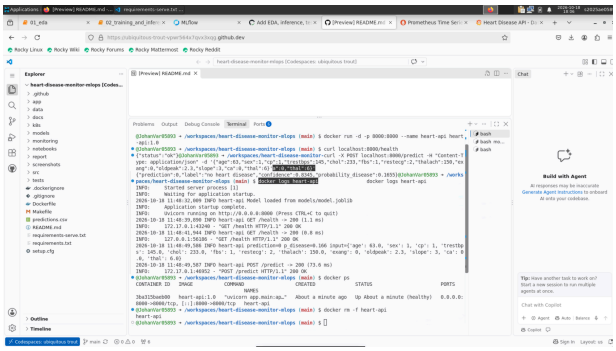


Figure 18. Calling /health and /predict on the running container.

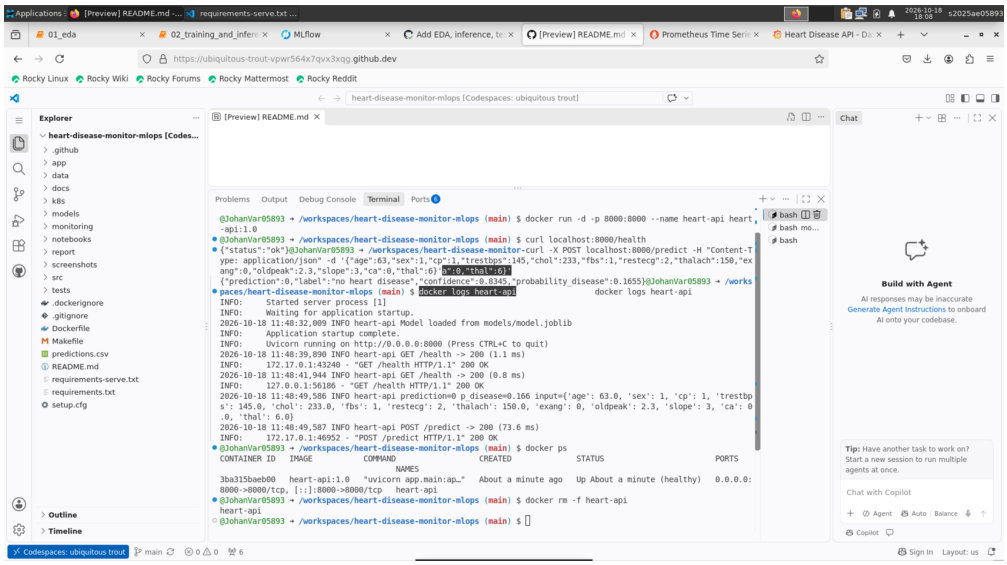


Figure 19. Container logs showing logged API requests.

9. Production deployment on Kubernetes

The manifests in k8s/ deploy the image as a Deployment with two replicas, readiness and liveness probes on /health, and CPU and memory requests and limits. A Service of type LoadBalancer exposes port 80 and forwards to container port 8000. The deployment was performed on a kind cluster inside a GitHub Codespace: both pods reached Running 1/1, and the API was verified through a port-forward. kind has no cloud load balancer, so the service shows EXTERNAL-IP as pending; on Docker Desktop, Minikube (with minikube tunnel) or a cloud cluster the same manifest receives an external address.

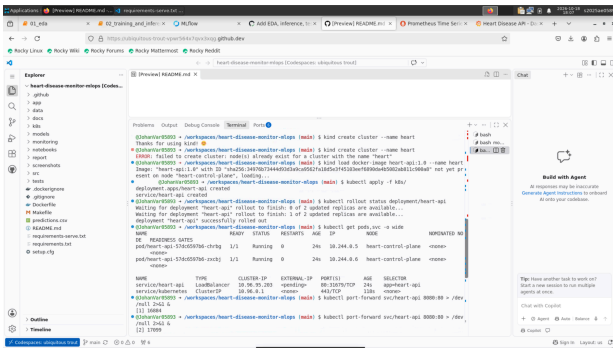


Figure 20. kubectl output: two heart-api pods Running and the LoadBalancer Service.

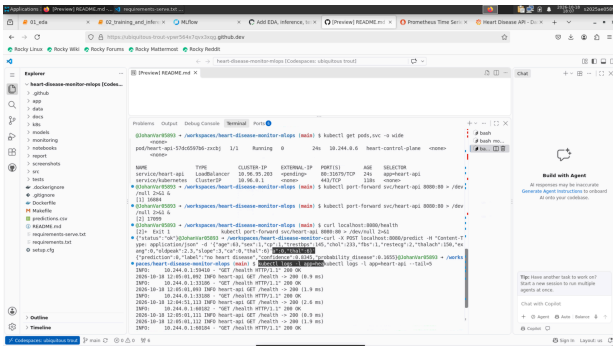


Figure 21. Calling /health and /predict on the Kubernetes deployment.

10. Monitoring and logging

Every request is logged with method, path, status and latency, and each prediction logs its input and result. The API exposes Prometheus metrics at /metrics: api_requests_total (by path, method and status), api_request_latency_seconds (histogram) and api_predictions_total (by predicted class). Docker Compose starts the API, Prometheus (scraping every 5 seconds) and Grafana, which is provisioned automatically with a Prometheus data source and a Heart Disease API dashboard.

Dashboard panel	Shows
Request rate by path	Traffic to /predict, /health and /metrics
Requests by status	200 vs 4xx responses (for example, 422 validation errors)
p95 latency (/predict)	95th percentile response time
Predictions by class	Count of predictions for each class
Total requests, error ratio, API up	Headline health indicators

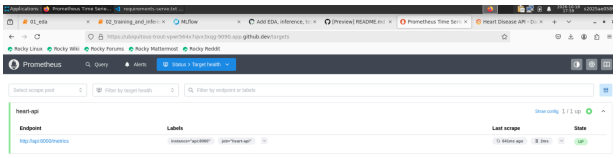


Figure 22. Prometheus targets page: heart-api is UP.

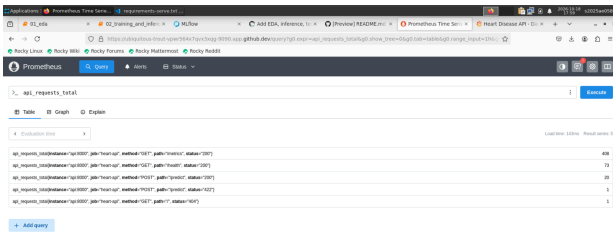


Figure 23. Prometheus query over the exported API metrics.

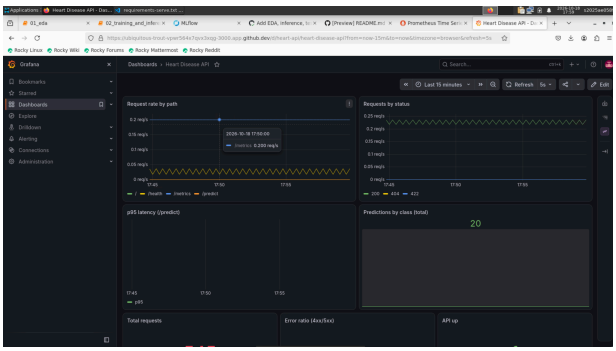


Figure 24. Grafana dashboard: request rate and status panels.

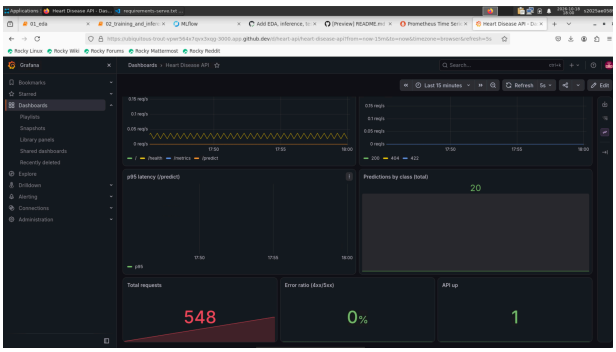


Figure 25. Grafana dashboard: total requests, 0% error ratio, API up, predictions by class.

11. Requirements coverage and repository structure

Assignment task	Where it is covered
1. Data acquisition and EDA	Section 3; src/data.py, src/eda.py, notebooks/01_eda.ipynb
2. Feature engineering and models	Section 4; src/features.py, src/train.py
3. Experiment tracking	Section 5; MLflow (mlflow.db), src/train.py
4. Packaging and reproducibility	Section 6; models/, requirements.txt, requirements-serve.txt
5. CI/CD and automated testing	Section 7; tests/, .github/workflows/ci.yml
6. Containerisation	Section 8; Dockerfile, app/main.py
7. Production deployment	Section 9; k8s/deployment.yaml, k8s/service.yaml
8. Monitoring and logging	Section 10; monitoring/, request logging in app/main.py
9. Documentation and reporting	This report; README.md; docs/architecture.png

Path	Purpose
src/	Data download and cleaning, EDA, features, training, inference
app/main.py	FastAPI service with metrics and logging
tests/	13 pytest unit tests
notebooks/	EDA notebook and training/inference notebook
.github/workflows/	CI pipeline (ci.yml) and a manual deployment demo workflow
Dockerfile, requirements*.txt	Container image and pinned dependencies
k8s/, monitoring/	Kubernetes manifests; Prometheus, Compose and Grafana provisioning
docs/, report/, screenshots/	Architecture diagram, this report, evidence images

12. Conclusion and future work

The project delivers a reproducible path from raw data to a monitored, containerised model: tracked experiments, tested and linted code, a CI pipeline that fails on errors, a Docker image served through FastAPI, a two-replica Kubernetes deployment and a live Grafana dashboard. Natural extensions are a model registry with staged promotion, drift monitoring on inputs and predictions, Helm packaging, automated deployment from CI, and validation on larger multi-site data.

Code repository: https://github.com/JohanVar05893/heart-disease-monitor-mlops

Pipeline video:
https://drive.google.com/file/d/1JQYdvKA8mJlrv3UU5GVnvCiacO9p3CyI/view?usp=drive_link